

Análise de Algoritmos de Ordenação em um Sistema de Ranking de Jogadores

Carlos Eduardo, Leôncio Ferreira, Paulo Gabriel, Cícero Jesus, Alan Mendes, Salomão Rodrigues

Resumo—Este trabalho investiga o impacto da escolha de estruturas de dados e algoritmos de ordenação no desempenho de um sistema de ranking de jogadores em jogos online. Foram implementados e avaliados os algoritmos Bubble Sort, Insertion Sort, Selection Sort, Quick Sort e Merge Sort utilizando duas estruturas distintas: lista encadeada estática e lista encadeada dinâmica. Os experimentos foram conduzidos considerando diferentes volumes de dados (100, 1000 e 10000 jogadores) e diferentes tipos de entrada (ordenada, aleatória e inversamente ordenada). Os resultados foram obtidos por meio de experimentos repetidos e análise do tempo médio de execução, permitindo comparar o comportamento dos algoritmos em diferentes cenários. A análise empírica demonstrou que algoritmos de complexidade quadrática apresentam desempenho significativamente inferior para grandes volumes de dados, enquanto Quick Sort e Merge Sort apresentam melhor escalabilidade. Entre os algoritmos avaliados, o Merge Sort demonstrou comportamento mais consistente e previsível, mantendo complexidade $O(n \log n)$ em todos os casos. Dessa forma, conclui-se que o Merge Sort representa a solução mais adequada para sistemas de ranking dinâmicos que manipulam grandes volumes de dados.

I. INTRODUÇÃO

Atualmente, os jogos online competitivos apresentam um crescimento expressivo em escala global [1]. A prática moderada desses jogos tem sido associada a benefícios cognitivos, tais como a melhoria do tempo de reação [2], atenção visual [2], agilidade na tomada de decisão [3] e aumento da capacidade de multitarefa [4]. A grande maioria desses títulos competitivos baseia-se em um “sistema de ranking”, que consiste na classificação contínua dos jogadores fundamentada em métricas como pontuação geral, níveis alcançados ou tempo total de jogo. Esse sistema é essencial não apenas para fomentar a competitividade, mas também para garantir a qualidade da experiência do usuário, permitindo o pareamento com adversários de habilidades similares e fornecendo uma métrica clara de progressão pessoal.

Entretanto, manter um sistema de ranking atualizado representa um desafio computacional relevante. Em ambientes com grande quantidade de jogadores, novos registros podem ser adicionados continuamente, enquanto pontuações e posições precisam ser recalculadas com frequência. Nesse cenário, a eficiência dos algoritmos de ordenação e das estruturas de dados utilizadas torna-se um fator determinante para garantir desempenho adequado e atualizações rápidas do ranking.

Motivado por esse contexto, o presente trabalho tem como objetivo analisar o impacto da escolha de estruturas de dados e algoritmos de ordenação no desempenho de um sistema de ranking de jogos online. Trata-se de um ambiente computacionalmente exigente e altamente dinâmico, no qual novos jo-

gadores são adicionados e removidos rotineiramente, exigindo a reorganização constante das posições.

Para realizar essa investigação, foram selecionados os algoritmos de ordenação *Bubble Sort*, *Insertion Sort*, *Selection Sort*, *Quick Sort* e *Merge Sort*, implementados sob duas abordagens estruturais distintas: listas encadeadas estáticas e listas encadeadas dinâmicas. A metodologia baseia-se na coleta e análise de métricas de tempo de execução, avaliando o comportamento de cada algoritmo submetido a diferentes tipos de entrada (ordenada, inversamente ordenada e aleatória) e variados volumes de dados. Essa abordagem permite uma comparação empírica rigorosa, visando identificar a solução algorítmica e estrutural mais adequada, eficiente e escalável para o cenário proposto.

II. DESENVOLVIMENTO

Com base nos experimentos realizados e na análise comparativa entre os algoritmos de ordenação implementados (Bubble Sort, Insertion Sort, Selection Sort, Quick Sort e Merge Sort), o algoritmo *Merge Sort* foi identificado como o mais adequado para o problema de ordenação de jogadores no sistema de ranking proposto.

Nos experimentos realizados foram considerados diferentes volumes de dados (100, 1000 e 10000 jogadores), bem como diferentes tipos de entrada (ordenada, aleatória e inversamente ordenada). Esses cenários foram escolhidos para simular situações reais de um sistema de ranking de jogos online, no qual novos jogadores podem ser inseridos continuamente e os dados precisam ser reorganizados com frequência.

Os resultados obtidos demonstraram que algoritmos de complexidade quadrática, como Bubble Sort, Insertion Sort e Selection Sort, apresentam desempenho inferior para grandes volumes de dados, conforme discutido por Cormen et al. [5]. Embora apresentem desempenho aceitável para conjuntos pequenos de dados, esses algoritmos tornam-se significativamente mais lentos à medida que o volume de elementos aumenta. Em contraste, algoritmos mais eficientes, como Quick Sort e Merge Sort, apresentam melhor escalabilidade e desempenho geral.

Entre os algoritmos avaliados, o Merge Sort demonstrou comportamento mais consistente e previsível em todos os cenários analisados, o que motivou sua escolha como algoritmo ideal para o problema proposto.

A. Descrição do Algoritmo Merge Sort

O Merge Sort é um algoritmo de ordenação baseado no paradigma *divide and conquer*, conforme descrito por Cormen

et al. [5]. Esse paradigma consiste em dividir um problema maior em subproblemas menores, resolver cada um deles individualmente e, posteriormente, combinar as soluções obtidas.

O funcionamento do Merge Sort pode ser dividido em três etapas principais, conforme apresentado por Cormen et al. [5]:

- **Divisão (Divide):** A lista de elementos é recursivamente dividida em duas sublistas aproximadamente iguais até que cada sublista contenha apenas um elemento.
- **Conquista (Conquer):** Cada sublista é tratada como uma lista ordenada individualmente.
- **Combinação (Merge):** As sublistas ordenadas são mescladas em uma única lista também ordenada.

No contexto do sistema de ranking desenvolvido neste trabalho, o algoritmo é utilizado para ordenar os jogadores com base em critérios como pontuação, nível ou tempo de jogo.

B. Pseudocódigo

O pseudocódigo do algoritmo Merge Sort apresentado a seguir foi adaptado a partir da implementação disponível em [6].

Algorithm 1 Merge Sort

```

1: mergesort(A[0...n-1], inicio, fim)
2: se (inicio < fim)
3:   meio ← (inicio + fim)/2
4:   mergesort(A, inicio, meio)
5:   mergesort(A, meio + 1, fim)
6:   merge(A, inicio, meio, fim)
7: fim_se

```

O pseudocódigo apresentado considera que A representa a estrutura de dados que armazena os jogadores a serem ordenados (por exemplo, um vetor ou lista). O valor n corresponde ao número total de elementos dessa estrutura. Os parâmetros $inicio$ e fim indicam, respectivamente, as posições inicial e final do segmento atualmente considerado pelo algoritmo durante o processo recursivo de divisão da lista.

Algorithm 2 Merge

```

1: merge(A, inicio, meio, fim)
2: criar vetores auxiliares Esq e Dir
3: copiar elementos de A[inicio..meio] para Esq
4: copiar elementos de A[meio + 1..fim] para Dir
5: i ← 0, j ← 0, k ← inicio
6: enquanto (i < tamEsq) e (j < tamDir)
7:   se (Esq[i] < Dir[j])
8:     A[k] ← Esq[i] ; i ← i + 1
9:   senão
10:    A[k] ← Dir[j] ; j ← j + 1
11:   k ← k + 1
12: fim_enquanto
13: copiar elementos restantes de Esq ou Dir para A

```

C. Análise de Complexidade

Uma das principais vantagens do Merge Sort é que sua complexidade de tempo é consistente em diferentes cenários de entrada.

Segundo Cormen et al. [5], o algoritmo Merge Sort apresenta as seguintes complexidades de tempo:

Melhor caso: $O(n \log n)$

Caso médio: $O(n \log n)$

Pior caso: $O(n \log n)$

Durante os experimentos realizados, observou-se que em alguns cenários específicos o Quick Sort apresentou tempos de execução ligeiramente menores. Entretanto, o Merge Sort demonstrou comportamento mais consistente entre os diferentes volumes de dados e tipos de entrada analisados.

Essa previsibilidade, aliada à sua complexidade estável, torna o Merge Sort uma escolha robusta para aplicações que exigem ordenação eficiente de grandes volumes de dados.

III. RESULTADOS E DISCUSSÃO

Esta seção apresenta e discute os resultados experimentais obtidos com a aplicação dos cinco algoritmos de ordenação — Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort — sobre dados de jogadores

A. Metodologia

A metodologia adotada compreendeu três etapas principais. Primeiramente, todos os algoritmos de ordenação foram implementados na linguagem C, tendo como base os pseudocódigos apresentados em Cormen et al. [5]

Em seguida, aplicou-se um algoritmo de *benchmark* que executa cada experimento 100 vezes, utilizando a média dos tempos coletados como métrica de análise.

Por fim, os resultados obtidos foram utilizados para a geração dos gráficos apresentados nas Figuras 1 e 2, por meio das bibliotecas Pandas e Matplotlib.

Os experimentos foram conduzidos em um notebook Lenovo LOQ 15IAX9E, equipado com processador Intel Core i5-12450HX (8 núcleos, frequência máxima de 4,40 GHz) e 16 GB de memória RAM, sob o sistema operacional Linux Mint 22.2 (base Ubuntu 24.04), com kernel 6.14.0-37.

Geração de Dados

Os registros de jogadores utilizados nos experimentos foram gerados sinteticamente em C por meio da função `rand()`, simulando um ambiente de ranking online. Cada registro contém *nickname* formado por um prefixo concatenado a um índice sequencial, *pontuacao* e *nivel* sorteados uniformemente no intervalo $[0, 10000]$, e *tempo_jogado* sorteado no intervalo $[0, 5000]$ horas. Para os cenários de entrada ordenada e inversamente ordenada, os registros foram previamente organizados pelo critério de pontuação antes de serem fornecidos ao algoritmo avaliado, reproduzindo, respectivamente, o melhor e o pior caso de algoritmos como Insertion Sort e Bubble Sort.

Medição do Tempo de Execução

O tempo de execução foi mensurado exclusivamente para o trecho correspondente ao algoritmo de ordenação, excluindo operações de geração de dados, entrada/saída e alocação

de memória. A medição foi realizada por meio da função `clock()` da biblioteca padrão `<time.h>` do C, que retorna o número de *ticks* de processador consumidos. O intervalo em milissegundos foi obtido pela expressão:

$$t_{ms} = \frac{clock_{fim} - clock_{inicio}}{CLOCKS_PER_SEC} \times 1000 \quad (1)$$

Cada configuração foi executada 100 vezes de forma independente, e o tempo médio foi adotado como resultado final, garantindo a suavização de variações causadas pelo escalonador do sistema operacional.

B. Análise Comparativa — Estrutura Estática

Os resultados na estrutura de lista encadeada estática estão apresentados na Figura 1. A escala logarítmica no eixo vertical permite a visualização simultânea das diferenças de desempenho entre algoritmos cujos tempos variam em ordens de magnitude.

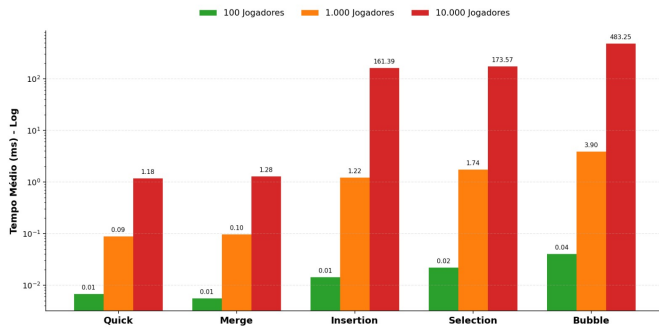


Figura 1. Tempo médio de execução (ms, escala log) dos algoritmos de ordenação na estrutura estática para 100, 1.000 e 10.000 jogadores.

Quick Sort e Merge Sort: Eficiência Assintótica Confirmada

O Quick Sort e o Merge Sort demonstraram, consistentemente, os menores tempos de execução na estrutura estática. Para 10.000 elementos, o Quick Sort registrou 1,18 ms e o Merge Sort 1,28 ms, enquanto o Bubble Sort atingiu 483,25 ms — diferença de aproximadamente 400 vezes.

Cormen et al. [5] estabelecem que o Quick Sort, apesar de possuir pior caso $\Theta(n^2)$, na prática apresenta tempo esperado de execução $\Theta(n \log n)$.

“O algoritmo quicksort tem tempo de execução do pior caso de $\Theta(n^2)$ para um arranjo de entrada de n números. Apesar desse tempo de execução lento para o pior caso, muitas vezes, o quicksort é a melhor opção prática para ordenação, devido à sua notável eficiência na média.”

O Merge Sort, por sua vez, garante $\Theta(n \log n)$ mesmo no pior caso.

Bubble Sort, Insertion Sort e Selection Sort: Degradação Quadrática

Os algoritmos de complexidade $\Theta(n^2)$ — Bubble Sort, Selection Sort e Insertion Sort — apresentaram crescimento expressivo dos tempos de execução ao passar de 1.000 para

10.000 elementos, confirmando o comportamento quadrático previsto teoricamente.

Cormen et al. [5] classificam o Bubble Sort como um algoritmo simples, porém ineficiente.

“O bubblesort é um algoritmo de ordenação popular, porém ineficiente. Ele funciona permutando repetidamente elementos adjacentes que estão fora de ordem.”

Considerações

A tabela de complexidades apresentada por Cormen et al. [5] resume formalmente o comportamento de cada algoritmo quanto ao pior caso e ao caso médio.

C. Análise Comparativa — Estrutura Dinâmica

Os resultados na estrutura de lista encadeada dinâmica estão apresentados na Figura 2. A hierarquia de desempenho entre os algoritmos se manteve semelhante à estrutura estática, porém com variações notáveis nos tempos absolutos.

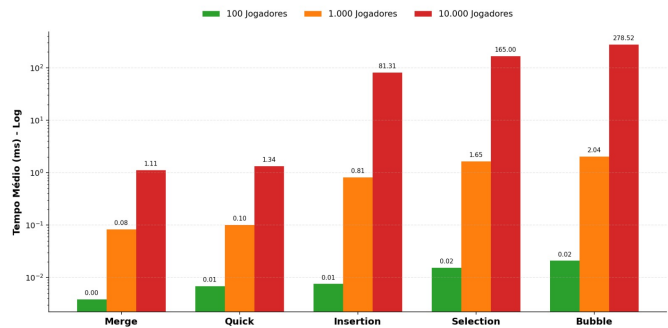


Figura 2. Tempo médio de execução (ms, escala log) dos algoritmos de ordenação na estrutura dinâmica para 100, 1.000 e 10.000 jogadores.

Merge Sort e Quick Sort na Estrutura Dinâmica

Na estrutura dinâmica, o Merge Sort obteve o melhor resultado com 10.000 elementos: 1,11 ms, ligeiramente inferior ao Quick Sort (1,34 ms). Ambos permaneceram muito superiores aos demais, com o Bubble Sort registrando 278,52 ms — redução em relação à estrutura estática (483,25 ms), porém ainda expressivamente maior que os algoritmos $\Theta(n \log n)$.

O Insertion Sort apresentou desempenho de 81,31 ms para 10.000 elementos na estrutura dinâmica, contra 161,39 ms na estática — redução de aproximadamente 50%. Esse ganho relativo deve-se ao fato de que a lista encadeada dinâmica permite inserções e remoções em $O(1)$ com uso de ponteiros, favorecendo algoritmos que fazem muitas operações de movimentação de elementos.

Conforme Cormen et al. [5], a lista encadeada fornece representação flexível para conjuntos dinâmicos, dando suporte a inserções e remoções eficientes:

“Listas ligadas nos dão uma representação simples e flexível para conjuntos dinâmicos, suportando todas as operações [...]. O tempo de execução para LIST-INSERT para uma lista de n elementos é $O(1)$.”

D. Impacto da Estrutura de Dados no Desempenho

Uma análise cruzada entre as Figuras 1 e 2 revela que a estrutura dinâmica, para a maioria dos algoritmos, apresentou tempos iguais ou menores para grandes volumes de dados. Essa vantagem é especialmente pronunciada nos algoritmos quadráticos (Bubble, Insertion e Selection), enquanto para Quick Sort e Merge Sort a diferença foi menos expressiva.

Para entradas pequenas (100 elementos), todos os algoritmos convergiram para tempos similares (abaixo de 0,04 ms), independentemente da estrutura utilizada. Isso é coerente com a análise teórica: para n pequeno, as constantes ocultas na notação assintótica dominam sobre a diferença de complexidade. Cormen et al. [5] alertam que fatores constantes podem ser relevantes para entradas pequenas:

“A ordenação por inserção normalmente tem um fator constante menor que a ordenação por intercalação [...]. Embora a ordenação por inserção em geral seja mais rápida que a ordenação por intercalação para pequenos tamanhos de entradas, tão logo o tamanho da entrada n se torne grande o suficiente, a vantagem da ordenação por intercalação de $\log n$ contra n compensará com sobras a diferença em fatores constantes.”

Para contextos onde os dados crescem dinamicamente — como um ranking de jogadores em tempo real, em que jogadores entram e saem continuamente — a estrutura dinâmica apresenta vantagem adicional de flexibilidade de memória, não exigindo realocação de blocos contíguos.

E. Algoritmo Ideal para o Caso Real

Considerando os dados empíricos obtidos e a análise teórica fundamentada em Cormen et al. [5], o Merge Sort é o algoritmo mais adequado para a ordenação do ranking de jogadores. Seus tempos de execução foram rivalizados com os do Quick Sort, com comportamento estável em ambas as estruturas de dados avaliadas.

Além do desempenho empírico, o Merge Sort apresenta como vantagem sua confiabilidade. Enquanto o Quick Sort tem como pior caso $\Theta(n^2)$ — quando o pivô definido não representa corretamente um valor mediano na régua numérica dos valores desordenados —, o Merge Sort se mantém constante em $\Theta(n \log n)$, sendo mais confiável em um sistema de inserções constantes.

Cormen et al. [5] destacam essa característica:

“O tempo de execução do quicksort depende de o particionamento ser balanceado ou não balanceado, o que por sua vez depende de quais elementos são usados para particionar. Se o particionamento é balanceado, o algoritmo é executado assintoticamente tão rápido quanto a ordenação por intercalação. Contudo, se o particionamento é não balanceado, ele pode ser executado assintoticamente tão lento quanto a ordenação por inserção.”

Os algoritmos quadráticos (Bubble Sort, Insertion Sort e Selection Sort) mostraram-se inviáveis para volumes de dados

acima de 1.000 elementos, com tempos que inviabilizariam uma atualização de ranking em tempo aceitável para o usuário final.

IV. CONCLUSÃO

Este artigo investigou o desempenho de diferentes algoritmos de ordenação aplicados a um sistema de ranqueamento de jogadores, avaliando também o impacto da utilização de listas encadeadas estáticas e dinâmicas. A análise empírica demonstrou que, embora o algoritmo Quick Sort tenha apresentado um tempo de execução inferior ao Merge Sort em um cenário específico, essa diferença limitou-se a uma margem de poucos nanossegundos. Em contrapartida, o Merge Sort obteve resultados consistentemente superiores ou equivalentes em todos os demais cenários propostos.

A escolha do Merge Sort como algoritmo ideal para esta aplicação fundamenta-se em um *trade-off* consciente entre pico de velocidade isolado e previsibilidade estrutural. Conforme descrito por Cormen et al. [5], o Merge Sort garante uma complexidade assintótica de tempo de $\Theta(n \log n)$ para o pior, melhor e médio casos, eliminando o risco de degradação para $O(n^2)$ inerente ao Quick Sort.

Além de sua eficiência matemática, o Merge Sort destaca-se por ser um algoritmo estável. No contexto de um *leaderboard* de jogos eletrônicos, onde novos registros e atualizações ocorrem continuamente, a estabilidade garante que jogadores com pontuações idênticas mantenham a ordem cronológica de suas conquistas. Por fim, ressalta-se que a mecânica de divisão e conquista do Merge Sort adapta-se de maneira excepcionalmente eficiente a listas encadeadas (dinâmicas e estáticas), uma vez que a junção (*merge*) das sublistas pode ser realizada apenas com a reatribuição de ponteiros, sem a necessidade de acessos aleatórios diretos à memória que favorecem outros algoritmos.

Portanto, os resultados empíricos aliados à análise teórica reafirmam que o Merge Sort oferece o equilíbrio ideal entre escalabilidade, segurança contra piores casos e confiabilidade, sendo a escolha mais robusta para a ordenação de rankings em tempo real.

REFERÊNCIAS

- [1] S. Kelly and S. Leung, “The new frontier of esports and gaming: A scoping meta-review of health impacts and research agenda,” *Frontiers in Sports and Active Living*, vol. 3, 2021. [Online]. Available: <https://www.frontiersin.org/journals/sports-and-active-living/articles/10.3389/fspor.2021.640362/full>
- [2] C. S. Green and D. Bavelier, “Action video game modifies visual selective attention,” *Nature*, vol. 423, no. 6939, pp. 534–537, 2003. [Online]. Available: <https://www.nature.com/articles/nature01647>
- [3] C. S. Green, A. Pouget, and D. Bavelier, “Improved probabilistic inference as a general learning mechanism with action video games,” *Current Biology*, vol. 20, no. 17, pp. 1573–1579, 2010. [Online]. Available: [https://www.cell.com/current-biology/fulltext/S0960-9822\(10\)00942-5?script=true](https://www.cell.com/current-biology/fulltext/S0960-9822(10)00942-5?script=true)
- [4] J. A. Anguera et al., “Video game training enhances cognitive control in older adults,” *Nature*, vol. 501, no. 7465, pp. 97–101, 2013. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/24005416/>
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Algoritmos: Teoria e Prática*, 3rd ed. Elsevier, 2012.
- [6] Cyberini, “Merge sort,” 2018, acesso em: 14 mar. 2026. [Online]. Available: <https://www.blogcyberini.com/2018/07/merge-sort.html>